Modul 2.3: Verzeichnisse und Dateien anlegen

Anschluss an Modul 2.2

In Modul 2.2 hast du den Linux-Dateisystembaum erkundet und mit `pwd`, `ls`, `cd` und verschiedenen Pfaden gearbeitet. Jetzt baust du selbst eine Projektstruktur auf und veränderst sie gezielt.

Du arbeitest ausschließlich in deinem Home-Verzeichnis. Alle Ordner und Dateien, die du für dieses Modul brauchst, legst du selbst an. So kannst du die Übungen auf jedem Ubuntu-Laptop wiederholen.

Lernziele und Praxisbezug

Nach diesem Modul kannst du:

- Verzeichnisse mit `mkdir` und `mkdir -p` erstellen,
- leere Dateien mit `touch` anlegen,
- Dateien und Verzeichnisse mit `cp` kopieren,
- Dateien mit `mv` verschieben und umbenennen,
- Dateien und Verzeichnisse mit `rm` löschen,
- deine Arbeit mit `tree` übersichtlich anzeigen,
- eine eigene Projektstruktur nach Anforderungen aufbauen.

Eine klare Struktur hilft allen, die an einem Projekt arbeiten. Häufig stehen Quellcode in `src/`, Tests in `tests/`, Dokumentation in `docs/` und Konfigurationen in `config/`.

> **Caution**
>
> Verwende alle schreibenden und löschenden Befehle in diesem Modul nur für `~/vorlagen`, `~/webprojekt` und `~/api-projekt`. `rm` löscht ohne Papierkorb. Prüfe vor dem Löschen immer den vollständigen Pfad.

1. Ein erstes Verzeichnis mit mkdir

`mkdir` steht für **make directory** und erstellt ein Verzeichnis. Lege dein Webprojekt an:

```
mkdir ~/webprojekt
```

Der Befehl gibt bei Erfolg normalerweise nichts aus. Das ist bei Linux beabsichtigt: Keine Fehlermeldung bedeutet in vielen Fällen, dass der Befehl erfolgreich war.

Prüfe selbst, ob das Verzeichnis existiert:

```
ls ~
```

In der Ausgabe sollte `webprojekt` stehen. Wenn du `mkdir: File exists` siehst, existiert der Ordner bereits. Prüfe ihn mit:

```
ls -la ~/webprojekt
```

Lösche einen Ordner nicht einfach. Er kann zu einem früheren Durchlauf gehören.

2. Verschachtelte Verzeichnisse mit mkdir -p

Ein Projekt besteht oft aus mehreren Ebenen. Probiere den folgenden Befehl aus:

```
mkdir ~/webprojekt/src/components/buttons
```

Du erhältst wahrscheinlich eine Fehlermeldung wie `No such file or directory`. Die übergeordneten Verzeichnisse `src` und `components` gibt es noch nicht. Ohne zusätzliche Option erstellt `mkdir` nur die letzte Ebene.

Mit `-p` werden fehlende übergeordnete Verzeichnisse automatisch angelegt:

```
mkdir -p ~/webprojekt/src/components/buttons
mkdir -p ~/webprojekt/tests/unit
```

Prüfe die Struktur:

```
tree ~/webprojekt
```

`tree` zeigt Verzeichnisse und Dateien als Baum. Falls `tree` nicht vorhanden ist, kannst du die Struktur alternativ mit diesem bereits vorhandenen Befehl anzeigen:

```
find ~/webprojekt -print
```

`mkdir -p` kannst du auch für einen bereits vorhandenen Pfad verwenden. In diesem Fall entsteht kein Fehler.

3. Leere Dateien mit touch

`touch` legt eine leere Datei an, wenn sie noch nicht existiert. Erzeuge die ersten Dateien für dein Webprojekt:

```
touch ~/webprojekt/src/index.html
touch ~/webprojekt/src/style.css
touch ~/webprojekt/README.md
```

Prüfe die Dateien:

```
ls -l ~/webprojekt/src
ls -l ~/webprojekt
```

In der langen Ausgabe steht am Anfang:

- `d` für ein Verzeichnis,
- `-` für eine normale Datei.

Die Dateigröße beträgt zunächst 0 Bytes, weil die Dateien noch keinen Inhalt haben. Später kannst du sie mit einem Editor bearbeiten.

Leere Dateien können als Platzhalter dienen. Eine Datei namens `.gitkeep` wird häufig als Marker verwendet, damit ein ansonsten leeres Verzeichnis in einem Versionskontrollprojekt sichtbar bleibt. Punktdateien sind versteckt. Du kannst sie mit `tree -a` anzeigen.

4. Dateien mit cp kopieren

`cp` steht für **copy**. Beim Kopieren bleibt das Original erhalten. Du erzeugst für diese Übung zuerst selbst einen Vorlagenbereich:

```
mkdir -p ~/vorlagen/config
touch ~/vorlagen/app.conf
touch ~/vorlagen/config/settings.conf
```

Prüfe die Vorlagen:

```
ls ~/vorlagen
tree ~/vorlagen
```

Eine einzelne Datei kopieren

Kopiere `app.conf` in dein Webprojekt:

```
cp ~/vorlagen/app.conf ~/webprojekt/app.conf
```

Prüfe, dass Original und Kopie existieren:

```
ls ~/vorlagen/app.conf ~/webprojekt/app.conf
```

Die Datei unter `~/vorlagen/` bleibt unverändert.

Ein Verzeichnis kopieren

Ein Verzeichnis kopierst du mit der Option `-r`. Das bedeutet **recursive**: Der gesamte Inhalt wird mitkopiert.

```
cp -r ~/vorlagen/config ~/webprojekt/config
```

Prüfe das Ergebnis:

```
tree ~/webprojekt/config
```

Die beiden grundlegenden Varianten:

Befehl	Kopiert	Beispiel
`cp quelle ziel`	eine Datei	`cp ~/vorlagen/app.conf ~/webproj`
`cp -r quelle z`	ein Verzeichnis samt Inhalt	`cp -r ~/vorlagen/config ~/webpro`

Prüfe vor dem Kopieren immer, ob Quelle und Ziel so heißen, wie du erwartest.

5. Verschieben und Umbenennen mit mv

Die kopierte Datei `app.conf` liegt momentan direkt in `~/webprojekt/`. Konfigurationsdateien sollen aber im Verzeichnis `config/` liegen und künftig `app.cfg` heißen.

`mv` steht für **move**. Verschiebe und benenne die Datei in einem Schritt um:

```
mv ~/webprojekt/app.conf ~/webprojekt/config/app.cfg
```

Prüfe den neuen Ort:

```
ls ~/webprojekt/config
```

Prüfe außerdem, dass die Datei am alten Ort nicht mehr liegt:

```
ls ~/webprojekt/app.conf
```

Hier erscheint eine Fehlermeldung wie `No such file or directory`. Das ist erwartet: Bei `mv` bleibt am ursprünglichen Ort keine Kopie zurück.

Linux benötigt keinen eigenen Umbenennen-Befehl:

Aktion	Beispiel
Verschieben	`mv datei.txt ordner/`
Umbenennen	`mv alter-name.txt neuer-name.txt`
Beides zusammen	`mv datei.txt ordner/neuer-name.txt`

Wenn du bei einem Ziel unsicher bist, prüfe zuerst mit `ls` oder `tree`. Verwende bei wichtigen Dateien zunächst `cp`, wenn du ein Sicherheitsnetz brauchst.

6. Dateien mit rm löschen

`rm` steht für **remove**. Es löscht Dateien endgültig. Linux-Server haben normalerweise keinen Papierkorb für diesen Befehl.

Erstelle zuerst einen ausschließlich für diese Übung gedachten Ordner:

```
mkdir ~/webprojekt/temp
touch ~/webprojekt/temp/notizen.txt
touch ~/webprojekt/temp/debug.log
```

Lösche eine einzelne Datei:

```
rm ~/webprojekt/temp/notizen.txt
```

Prüfe das Ergebnis:

```
ls ~/webprojekt/temp
```

Nur `debug.log` sollte noch vorhanden sein.

Ein Verzeichnis rekursiv löschen

Versuche zunächst, das leere Verzeichnis mit dem normalen Befehl zu löschen:

```
rm ~/webprojekt/temp
```

`rm` verweigert das normalerweise mit einem Hinweis wie `Is a directory`. Für Verzeichnisse und ihren gesamten Inhalt benötigst du `-r`:

```
rm -r ~/webprojekt/temp
```

Prüfe, dass das Verzeichnis verschwunden ist:

```
ls ~/webprojekt
```

Befehl	Wirkung	Sicherheitsaspekt
`rm datei`	löscht eine einzelne Datei	kein Papierkorb, kein Undo
`rm -r verzeich`	löscht Verzeichnis und Inhalt	alle Einträge im Unterbaum werden entfernt

Verwende in dieser Übung kein `rm -rf`. Die Option `-f` unterdrückt Rückfragen und ist für das Lernziel nicht erforderlich.

7. Die Struktur mit tree prüfen

Zeige dein Webprojekt vollständig an:

```
tree ~/webprojekt
```

Mit `-a` werden auch versteckte Dateien angezeigt:

```
tree -a ~/webprojekt
```

Du solltest unter anderem `README.md`, `src/`, `tests/`, `config/` und die verschobene Datei `config/app.cfg` sehen. Das temporäre Verzeichnis `temp/` wurde gelöscht.

Falls `tree` nicht vorhanden ist, verwende:

```
find ~/webprojekt -print
```

8. Selbstständige Aufgabe: Eine API-Struktur bauen

Baue jetzt ohne eine vorgegebene Befehlsfolge ein zweites Projekt. Erstelle unter `~/api-projekt` genau diese Struktur:

```
api-projekt/
├── src/
│   ├── controllers/
│   │   └── .gitkeep
│   ├── models/
│   │   └── .gitkeep
│   └── middleware/
│       └── .gitkeep
├── tests/
│   └── integration/
├── docs/
└── README.md
```

Überlege vor dem Tippen:

1. Welche Verzeichnisse kannst du mit einem oder mehreren `mkdir -p`- Befehlen anlegen?
2. Welche vier Dateien musst du mit `touch` erstellen?
3. Warum brauchst du bei den drei `.gitkeep`-Dateien den Punkt am Anfang?

Erlaubte Hilfsmittel:

```
mkdir --help
man mkdir
```

Prüfe dein Ergebnis:

```
tree -a ~/api-projekt
```

Falls `tree` fehlt:

```
find ~/api-projekt -print
```

Vergleiche deine Ausgabe mit dem Diagramm. Es müssen alle Verzeichnisse, `README.md` und die drei `.gitkeep`-Dateien vorhanden sein.

Transfer in den Arbeitsalltag

- **Systemintegration:** Prüfe vor einem rekursiven Löschen den Host, deinen Benutzer, das aktuelle Verzeichnis und den vollständigen Zielpfad.
- **Anwendungsentwicklung:** Vereinbare im Team feste Orte für Quellcode, Tests, Dokumentation und Konfiguration.
- **Daten- und Prozessanalyse:** Trenne beispielsweise Rohdaten in `data/raw/` von bearbeiteten Daten in `data/processed/`, damit Originale nicht versehentlich überschrieben werden.

Zusammenfassung

Du kannst jetzt:

- einzelne und verschachtelte Verzeichnisse anlegen,
- leere Dateien und versteckte Marker-Dateien erzeugen,
- Dateien und Verzeichnisse kopieren,
- Dateien verschieben und umbenennen,
- Dateien und eigene Übungsverzeichnisse löschen,
- eine Projektstruktur als Baum prüfen,
- Anforderungen in Verzeichnisse und Dateien übersetzen.

Bevor du Dateien verschiebst oder löschst, prüfe immer deinen aktuellen Standort, den vollständigen Pfad und die Wirkung des Befehls.